

Simulación de Entrevista Técnica — Domain-Driven Design (DDD)

Guía de estudio en formato entrevista · aplicación de **DDD y arquitectura

hexagonal** en el sistema POS (`venta-service` , `stock-service` , ... con capas `domain` / `application` / `infrastructure` / `interfaces`).

Cada sección incluye la **pregunta del entrevistador**, una **respuesta modelo**

anclada al código real de este proyecto y, cuando aplica, **preguntas de seguimiento** (follow-ups).

Formato sugerido: 45–60 min. Bloques: fundamentos (10'), building blocks

tácticos (15'), puertos y adaptadores (15'), diseño estratégico (10'), crítica/mejoras (10').

0. Warm-up — DDD en el proyecto

P ¿Cómo se refleja DDD en la estructura de este sistema POS?

R Cada microservicio organiza el paquete raíz (`com.venta` , `com.stock` , ...) en **cuatro capas** al estilo hexagonal: **domain** — el modelo de negocio: `model` (entidades como `Order` , `Product`), `event` , `port` (interfaces), `repository` , `exception` y `saga` ; **application** — casos de uso y orquestación: en `venta-service` hay `command` (`OrderCommandService`), `query` (`OrderQueryService`), `saga` (`SagaOrchestrator`) y `cart` (`CartService`); **infrastructure** — adaptadores técnicos: `kafka` (`VentaProducer` / `VentaConsumer`), `config` ; **interfaces/rest** — los controllers (`VentaController` , `CartController`). La dependencia apunta **hacia adentro**: `infrastructure` e `interfaces` conocen al `domain` , no al revés.

FOLLOW-UP ¿Es DDD "táctico" o "estratégico"? — Ambos: **táctico** en los building blocks (entidades, repositorios, puertos, eventos) y **estratégico** en que cada microservicio es un **bounded context** con su propia base de datos.

1. Fundamentos

P ¿Qué es un bounded context y cómo se materializa aquí?

R Un **bounded context** es una frontera dentro de la cual un modelo y su lenguaje ubicuo son consistentes. En este sistema cada **microservicio es un bounded context**: `venta-service` (ventas/órdenes/carrito), `stock-service` (inventario), `despacho-service` (despachos), `auth-service` (usuarios). Cada uno tiene su **base por servicio** (MongoDB para `venta/stock/despacho`, PostgreSQL para `auth`), así que el mismo concepto puede modelarse distinto en cada contexto sin acoplarse.

FOLLOW-UP ¿"Producto" es el mismo concepto en venta y en stock? — No exactamente: en `stock-service` `Product` tiene `sku`, `quantity`, `reservedQuantity`, `reservedByOrder`; en `venta-service` solo se referencia por `productId` dentro de `Order / CartItem`. Cada contexto modela lo que necesita.

P ¿Qué es el lenguaje ubicuo y dónde se ve en el código?

R Es el vocabulario compartido entre negocio y código. Aquí aparece en nombres de dominio: estados `PENDING`, `STOCK_RESERVED`, `DISPATCHING`, `COMPLETED`, `CANCELLED` (enum `Order.OrderStatus`), operaciones como `reserveStock`, `compensateStock`, `confirmStock` (interfaz `StockEventPublisher`), y estados de carrito `RESERVED / RESERVE_FAILED / RELEASED` (`CartItem`). Los métodos del service usan el idioma del negocio (`crearVenta`, `cancelarVenta`).

2. Building blocks tácticos

P Distingue Entity y Value Object. ¿Cuáles hay en el repo?

R Una **Entity** tiene identidad propia y ciclo de vida (dos entidades con los mismos datos pero distinto id son distintas). `Order` (`@Document("orders")` con `@Id String id`) y `Product` (`@Document("products")`) son entidades. Un **Value Object** se define solo por sus atributos, sin identidad, e idealmente es inmutable (p. ej. un `Money / Address`). En el proyecto el modelo es más plano: se usan tipos como `BigDecimal totalAmount` en `Order` en vez de un VO `Money` dedicado.

FOLLOW-UP ¿ `CartItem` es entity o value object? — Es una **entity** (tiene `@Id` y estado propio: `status`, `reservationId`, `expiresAt`), persistida en la colección `cart_items`.

P ¿Qué es un Aggregate y quién es la raíz aquí?

R Un **aggregate** es un grupo de objetos tratados como una unidad de consistencia, accedido solo por su **aggregate root**. En `venta-service`, `Order` actúa como raíz del agregado de la venta: las transiciones de estado y la consistencia de la orden se gestionan a través de ella (vía `OrderCommandService`). `Product` es la raíz del agregado de inventario en `stock-service`, y encapsula `reservedByOrder` (reservas por orden) para mantener su invariante de stock.

FOLLOW-UP ¿Por qué `Product.reservedByOrder` es un `Map<String,Integer>`? — Para hacer la reserva **idempotente y actualizable**: un mensaje Kafka reentregado con la misma cantidad da un delta cero, y al cambiar la cantidad solo se aplica el **delta** frente a lo ya reservado. Es una invariante del agregado documentada en el propio `Product`.

P ¿Cómo se protegen las invariantes de negocio?

R Con **excepciones de dominio** y guardas en la capa de aplicación. Por ejemplo, `OrderCommandService.cancelarVenta` lanza `InvalidOrderStateException` si la orden está `COMPLETED` o `CANCELLED` ("solo se puede cancelar lo que no es terminal"), y `OrderNotFoundException` si no existe. Son clases en `com.venta.domain.exception`, parte del modelo de dominio, no excepciones técnicas.

P ¿Qué patrón representan `OrderRepository` / `ProductRepository`?

R El patrón **Repository**: una abstracción de colección de agregados. Aquí son interfaces en `domain.repository` (`OrderRepository` extends `ReactiveMongoRepository<Order,String>`) con métodos con lenguaje de negocio como `findByStatus` o `findByStatusInAndUpdatedAtBefore` (usado por el `SagaReconciler` para detectar órdenes atascadas). El dominio depende de la **interfaz**; Spring Data provee la implementación en tiempo de ejecución.

3. Puertos y adaptadores (hexagonal)

P Explica cómo aplica el proyecto puertos y adaptadores.

R Los **puertos** son interfaces en `domain.port : StockEventPublisher ( reserveStock , compensateStock , confirmStock )` y `DespachoEventPublisher`. El **adaptador** es `VentaProducer` en `infrastructure.kafka`, que **implementa ambos puertos** y traduce las llamadas de dominio a `kafkaTemplate.send(...)` sobre topics concretos (`saga.stock.reserve-command` , etc.). Así el dominio expresa "*reserva stock*" sin saber que por debajo es Kafka: podrías cambiar a otro transporte reemplazando solo el adaptador.

FOLLOW-UP ¿Por qué esto es inversión de dependencias? — Porque la capa de aplicación (`OrderCommandService`) depende de la **abstracción** `StockEventPublisher` (inyectada por constructor con `@RequiredArgsConstructor`), no de la clase Kafka concreta. La flecha de dependencia queda invertida: la infraestructura depende del dominio.

P ¿Los `@Document` de MongoDB en las entidades no acoplan el dominio a la infraestructura?

R Sí, es una **concesión pragmática**: `Order` y `Product` llevan anotaciones de Spring Data (`@Document` , `@Id`) directamente en el modelo de dominio. Un DDD purista separaría la entidad de dominio del documento de persistencia (con un mapper). El proyecto elige simplicidad sobre pureza; el coste es que si cambiaras de MongoDB tendrías que tocar las clases de dominio.

FOLLOW-UP ¿Cómo lo harías "más puro"? — Con un modelo de dominio POJO puro y una clase `OrderDocument` en infraestructura, más un mapper en el adaptador de repositorio. Aumenta el boilerplate; se justifica cuando el dominio es rico y volátil.

P ¿Dónde encaja `SagaOrchestrator` en las capas?

R En `application.saga` : es un **caso de uso de orquestación**, no dominio puro. Coordina la transacción de negocio (reservar stock → pedir despacho → completar o compensar) usando los puertos y repositorios. La lógica de coordinación vive en aplicación; las reglas de cada agregado, en el dominio.

4. Diseño estratégico

P ¿Qué relación (context mapping) hay entre los bounded contexts?

R Se comunican por **eventos asíncronos vía Kafka**, un acoplamiento débil. El mapa es tipo **cliente-proveedor / eventos de integración**: `venta-service` (orquestrador SAGA) emite comandos/eventos que `stock-service` y `despacho-service` consumen y responden. No hay llamadas síncronas de dominio entre contextos; la integración es por mensajes, lo que preserva la autonomía de cada contexto.

P ¿Diferencia entre evento de dominio y evento de integración aquí?

R Un **evento de dominio** es algo relevante dentro de un contexto ("la orden pasó a `STOCK_RESERVED`"). Un **evento de integración** cruza fronteras entre contextos. En el repo los eventos que viajan por Kafka (`StockReserveEvent` , `DespachoRequestEvent` , `StockReserveResponseEvent`) son de **integración**, y se publican de forma fiable con el patrón **Outbox** (`OutboxEvent` + `OutboxPublisher`) para evitar el dual-write entre la base y el broker.

FOLLOW-UP ¿Por qué Outbox? — Para no perder eventos si el commit en Mongo tiene éxito pero el envío a Kafka falla (o viceversa): se escribe el evento en la misma transacción que el cambio de estado y un publisher lo despacha después.

5. Crítica y mejoras

P ¿Ves modelos de dominio anémicos? ¿Es un problema?

R Sí, hay tendencia anémica: `Order` , `Product` , `CartItem` usan Lombok (`@Data` / `@Builder`) y son básicamente estructuras de datos; la lógica de negocio (transiciones de estado, reglas de cancelación) vive en los *services* (`OrderCommandService`). En DDD idiomático esa lógica iría **dentro del agregado** (p. ej. `order.cancel()` que valide el estado). No es "malo" per se —es un estilo orientado a servicios— pero dispersa las invariantes y las hace más fáciles de saltarse.

FOLLOW-UP ¿Cómo lo enriquecerías sin romper todo? — Mover las guardas a métodos de la entidad (`Order.reserveStock()` , `Order.cancel()`), dejando el service como coordinador que orquesta persistencia y publicación de eventos.

P El estado se maneja con un enum plano `OrderStatus` . ¿Qué mejora propondrías?

R Modelarlo como **máquina de estados explícita** con transiciones válidas (PENDING → STOCK_RESERVED → DISPATCHING → COMPLETED; y ramas a STOCK_FAILED/ DISPATCH_FAILED/CANCELLED). Hoy la validez de transición se comprueba de forma dispersa (p. ej. el chequeo de cancelación). Centralizarla en la entidad evitaría estados imposibles y documentaría el ciclo de vida del agregado.

P Si tuvieras que enseñar la separación de capas a alguien nuevo, ¿qué regla darías?

R "Sigue las importaciones": una clase de `domain` **nunca** debe importar de `infrastructure` o `interfaces` . El dominio define **puertos** (interfaces) y la infraestructura los **implementa**. Si te ves importando `KafkaTemplate` o `ReactiveMongoRepository` desde el dominio, algo está en la capa equivocada.

Apéndice — Chuleta rápida

Concepto DDD	Dónde vive en el repo
Capas	<code>domain / application / infrastructure / interfaces</code> por servicio
Bounded context	Cada microservicio + su base propia (venta/stock/despacho = Mongo, auth = PG)
Aggregate root	<code>Order</code> (venta), <code>Product</code> (stock)
Entity	<code>Order</code> , <code>Product</code> , <code>CartItem</code> (<code>@Document</code> + <code>@Id</code>)
Repository	<code>OrderRepository</code> / <code>ProductRepository</code> (interfaces en <code>domain.repository</code>)
Puerto	<code>StockEventPublisher</code> , <code>DespachoEventPublisher</code> (<code>domain.port</code>)
Adaptador	<code>VentaProducer</code> (<code>infrastructure.kafka</code>) implementa los puertos
Caso de uso	<code>OrderCommandService</code> (CQRS: command) / <code>OrderQueryService</code> (query)
Orquestación	<code>SagaOrchestrator</code> , <code>SagaReconciler</code> (<code>application.saga</code>)
Excepción de dominio	<code>InvalidOrderStateException</code> , <code>OrderNotFoundException</code>
Eventos integración	<code>StockReserveEvent</code> , <code>DespachoRequestEvent</code> + <code>Outbox</code> (<code>OutboxEvent</code>)
Concesión	Anotaciones Mongo en el dominio; modelo algo anémico (lógica en services)